RTL Design Sherpa

CDC Counter Display Nexys A7 Project Guide 0.90

July 2026

Table of Contents

List of Figures

List of Tables

List of Waveforms

1 Document Information

This is the operator / developer guide for the Nexys A7 **CDC Counter Display** project (`projects/NexysA7/cdc_counter_display`). It explains what the project is, how it works, how to build / simulate / program / run it, and how the UART characterization harness is configured. It is a practical guide, not a formal architecture specification.

The project ships in two phases that coexist in the same tree:

- **Phase 1** — a standalone, button-driven CDC counter (`cdc_counter_display_top`).
- **Phase 2** — a UART-controlled, four-counter CDC demonstrator (`cdc_demo_top`) with a host program that runs byte-for-byte identically on the FPGA and in a cocotb simulation. This guide focuses on Phase 2.

1.1 References

Document references

Source	Title
RTL Design Sherpa	`projects/NexysA7/cdc_counter_display/README.md`
RTL Design Sherpa	`docs/HARNESS.md` (CSR map + wiring)
RTL Design Sherpa	`docs/RUNBOOK.md` (operator procedures)
RTL Design Sherpa	`bin/TBClasses/harness/` (shared UART-char collateral)
Digilent	Nexys A7 Reference Manual
ARM	AMBA AXI and APB Protocol Specifications

1.2 Conventions

- **Registers are accessed by name**, never by hardcoded offset (see Chapter 4).
- **Clock:** `sys_clk` = 100 MHz on-board oscillator. Per-counter source clocks (`ctr_clk[i]`) are asynchronous to `sys_clk` and to each other.
- **Reset:** `CPU_RESETN` (board button BTNR, active-low) or `CTRL.soft_reset`.
- **Register/signal prefixes:** `r_` registered, `w_` combinational.

1.3 Revision History

Revision history

Version	Date	Notes
0.90	2026-07-18	Initial project guide

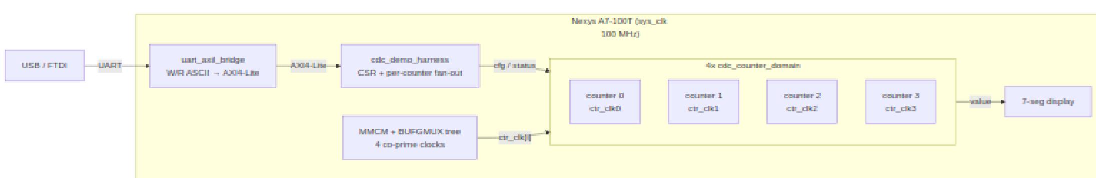
2 Overview

2.1 What it is

CDC Counter Display is an educational Nexys A7 project that demonstrates **clock domain crossing (CDC)** — both done correctly and broken on purpose — on real silicon. Four independent counters each run in their own asynchronous clock domain; a host program over UART configures them, reads their values back across the CDC boundary, and can deliberately select an unsafe crossing to *watch CDC fail* on the 7-segment display.

Phase 2 (`cdc_demo_top`) is wired on the standard Nexys A7 characterization **spine**: a single USB-UART link feeds a `uart_axil_bridge` that converts an ASCII `W`/`R` command protocol into AXI4-Lite, which drives the `cdc_demo_harness` CSR block. The harness fans configuration out to four `cdc_counter_domain` instances and collects their CDC'd status back.

2.1.1 Figure 1.1: CDC Demo Harness Spine



CDC demo harness spine

Source: 01_harness_spine.mmd

2.2 What it demonstrates

- **Safe multi-bit CDC.** With a Gray-coded or FIFO-based crossing, the host reads coherent counter values at any source-clock speed.
- **Broken CDC, visibly.** Put a counter in NO-CDC mode with auto-increment, then sweep its source clock from slow to fast: the display stays clean at low speeds and visibly scrambles at high speeds (multi-bit bus skew), while the other counters — in a safe mode — stay clean the whole time.
- **Five selectable CDC strategies** per counter: NO-CDC, stretch, sync-FIFO, two-phase handshake, four-phase handshake.
- **Sim / silicon equivalence.** The same host program drives the FPGA and a cocotb simulation over the identical UART byte stream (Chapter 6).

2.3 What you need

- Nexys A7-100T board (XC7A100T-1CSG324C) + USB cable.
- Xilinx Vivado (for `build-demo` / `program-demo`).
- A Python environment sourced from `env_python` (for the host CLI and sim).

2.4 Where things live

Project file layout

Path	Contents
`rtl/cdc_demo_top.sv`	Board top: clocking tree, buttons, harness, display
`rtl/cdc_demo_harness.sv`	AXI4-Lite CSR + per-counter fan-out
`rtl/cdc_counter_domain.sv`	One counter + its CDC paths (5 modes)
`rtl/cdc_demo_csr.rdl`	Register descriptor (generates the by-name regmap)
`host/`	`cdc_demo.py` driver, `cdc_programs.py`, `run_cdc_demo.py` CLI
`dv/`	UART-equivalence cocotb sim (tb, tests, filelist, regmap)
`Makefile`	Build / sim / program / host targets

3 How It Works

3.1 Top-level structure (`cdc_demo_top`)

The board top instantiates, from UART inward:

1. **`uart_axil_bridge`** — decodes the ASCII `W <addr> <data>` / `R <addr>` protocol into single-beat AXI4-Lite. `CLKS_PER_BIT` is derived from `SYS_CLK_HZ / 115200`.
2. **`cdc_demo_harness`** — the AXI4-Lite CSR slave (Chapter 5). Stores per-counter config, exposes CDC'd status, and generates single-cycle pulses (`CFG_LOAD`, `HOST_PRESS`).
3. **Clocking tree** — an `MMCME2_BASE` produces four co-prime divided clocks; per counter a `BUFGMUX_CTRL` tree glitchlessly selects one of five sources.
4. **Four `cdc_counter_domain` instances** — the counters and their CDC paths.
5. **Display** — `DISP_SELECT` picks which counter's value drives the 7-seg.

3.2 Clocking and the four source clocks

The MMCM synthesizes four **mutually asynchronous** clocks from the 100 MHz reference using pairwise co-prime divisors, plus a per-counter sys_clk-derived divided clock:

Per-counter clock sources

`clock_select`	Source	Frequency
0	MMCM CLKOUT0 (÷11)	72.7 MHz
1	MMCM CLKOUT1 (÷29)	27.6 MHz
2	MMCM CLKOUT2 (÷67)	11.9 MHz
3	MMCM CLKOUT3 (÷128)	6.25 MHz
4	`clock_divider` (uses `div_pickoff`)	`sys_clk / 2^(pickoff+1)`

The co-prime divisors `{11, 29, 67, 128}` mean no two outputs share an edge alignment for a very long time — for CDC purposes, truly asynchronous. The divided-clock branch (select 4) keeps a slow, visibly-counting rate available for the "watch it count" demo; `div_pickoff = 23` gives roughly 6 Hz.

3.3 One counter domain (`cdc_counter_domain`)

Each counter lives entirely in its `ctr_clk[i]` domain and crosses two ways:

- **sys_clk → ctr_clk (config in):** `INIT` / `INCREMENT` as quasi-static 2-FF synchronized buses; `CFG_LOAD` / `HOST_PRESS` as single-shot pulses through a `sync_pulse` module.
- **ctr_clk → sys_clk (status out):** `VALUE`, `PRESS_COUNT`, `CTR_CLK_TICKS` crossed back for readback. **The `VALUE` path is where `CDC_MODE` selects the crossing strategy** — this is the heart of the demo.

On each `ctr_clk` edge the counter loads `INIT` on a `CFG_LOAD` pulse, otherwise advances by `INCREMENT` when a press event occurs. A press event is a debounced button, a `HOST_PRESS` pulse, or — when `AUTO_INC = 1` — every `ctr_clk` edge.

3.4 The five CDC modes (`CDC_MODE`)

The five CDC modes for the VALUE readback path

Mode	Name	Primitive	Safe?
0	NO-CDC	raw flop per bit	No (multi-bit skew at fast clocks)
1	STRETCH	`cdc_open_loop` (pulse stretch + sync)	Up to a tuned cliff (~25 MHz)
2	SYNC-FIFO	`fifo_async` (Gray pointers)	Yes
3	TWO-PHASE	`cdc_2_phase_handshake`	Yes
4	FOUR-PHASE	`cdc_4_phase_handshake`	Yes

Mode 0 (NO-CDC) samples the multi-bit value with plain flops — at a fast source clock the bits arrive skewed and the host reads transient garbage. That is the intended failure. In Verilator (no metastability model) mode 0 reads the true value deterministically, which is why the simulation asserts on mode 0 for exact values (Chapter 6) but the *board* shows scramble at speed.

3.5 Display

`DISP_SELECT` chooses which counter's 16-bit `VALUE` drives the 7-segment digits; the upper digits show the selected counter's `CDC_MODE` and clock-select so the operator can read the current configuration off the board.

4 Test Methodology

This chapter describes what is exercised on the FPGA block (Figure 2.1) and how the results are judged. CDC Counter Display is a **demonstration**, so the “measurement” is a coherence test of the clock-domain-crossing readback rather than a throughput sweep.

4.1 What is under test

The four `cdc_counter_domain` value-readback paths, each selectable among five CDC strategies (`CDC_MODE` 0–4) and driven by an asynchronous source clock whose rate the host can sweep. The question under test: **does the multi-bit counter value cross into `sys_clk` coherently?**

4.2 What is measured, and how

What is measured

Quantity	How it is obtained
Value coherence	Read `VALUE` by name N times; compute min / max / number of unique samples
Press determinism	After `count` `HOST_PRESS` pulses, check `VALUE == INIT + count*INCREMENT` and `PRESS_COUNT` delta
Reload	`CFG_LOAD` reloads `VALUE` to `INIT` without disturbing `PRESS_COUNT`
Mode selection	Each `CDC_MODE` code round-trips through the CSR and selects a distinct path

A run configures a counter (mode, init, increment, clock), injects stimulus (`HOST_PRESS` or `AUTO_INC`), then samples `VALUE`/`PRESS_COUNT` over the by-name bridge. In the “watch-fail” workload the counter is put in NO-CDC with `AUTO_INC = 1` and its source clock is swept slow→fast; the **spread and unique count of the `VALUE` samples** is the metric — a few unique values means clean crossing, a wide 0x00–0xFF spread means multi-bit skew.

4.3 Workloads

Demonstration workloads

Workload	Purpose
`press`	Deterministic increment check (safe CDC mode)
`cfg_load`	Reload semantics
`cdc_mode`	All five mode codes round-trip

Workload	Purpose
`watch_fail`	Sweep an unsafe crossing slow→fast and watch it break

4.4 The oracle, and what sim can/can't show

The oracle is the arithmetic expectation (`INIT + presses*INCREMENT`) for the safe modes. In Verilator the NO-CDC path reads the *true* value (no metastability model), so the sim asserts exact values even in mode 0 — the analog "garbage" is a silicon-only effect. That asymmetry is the point of the demo, not a defect (Chapter 6, Sim Equivalence).

4.5 Measurement pitfalls

- NO-CDC garbage at speed is **expected** — use a safe `CDC_MODE` (2/3/4) before asserting exact values.
- `PRESS_COUNT` always uses Gray-coded CDC and stays coherent regardless of mode; prefer it when you need a trustworthy count.

5 Build and Run

All commands are run from `projects/NexysA7/cdc_counter_display/` after sourcing the Python environment:

```
cd /path/to/RTLDesignSherpa
source env_python            # sets SIM=verilator, PATH, PYTHONPATH,
REPO_ROOT
cd projects/NexysA7/cdc_counter_display
```

5.1 The workflow at a glance

```
make regmap  ->  make consistency  ->  make sim-demo   (prove it in
simulation)
                                                          |
                                                          v
             make build-demo  ->  make program-demo  ->
run_cdc_demo.py   (on the board)
```

5.2 Make targets

5.2.1 Simulation

Simulation make targets

Target	What it does
`make sim`	Phase-1 CocoTB sim of `cdc_counter_display_top`.
`make sim-demo`	Phase-2 **UART-equivalence** sim: wraps the real `uart_axil_bridge` + harness and runs the host programs over a cocotb UART master (`dv/tests/test_cdc_demo_uart.py`). Four tests: smoke, press, cfg_load, cdc_mode.

5.2.2 Register collateral

Register-collateral make targets

Target	What it does
`make regmap`	Regenerate `dv/tbclasses/cdc_demo_csr_regmap.py` from `rtl/cdc_demo_csr.rdl`. Run after editing the RDL — never hand-edit the regmap.
`make consistency`	Guard test: the generated regmap must match the hand-written `cdc_demo_harness.sv` (offsets + per-counter block).

5.2.3 Bitstream

Bitstream make targets

Target	What it does
`make build-demo`	Build the phase-2 bitstream `cdc_demo.bit` with Vivado (~5–10 min).
`make program-demo`	Flash the board with `cdc_demo.bit`.
`make lint-demo`	Verilator lint of the phase-2 RTL (Xilinx clocking primitives are stubbed — see Chapter 7).

5.3 The host CLI (`host/run_cdc_demo.py`)

The CLI builds a `CdcDemoDriver`, resolves the serial port (auto-probes every `/dev/ttyUSB*` for the `CDC1` build ID unless `--port` is given), and dispatches to the authored-once programs in `cdc_programs.py`.

```
# Verify the link and dump all four counters' defaults
python3 host/run_cdc_demo.py smoke

# Inject 1000 host presses to counter 2; checks VALUE = INIT +
1000*INCREMENT
python3 host/run_cdc_demo.py press --counter 2 --count 1000

# CFG_LOAD reload check / CDC_MODE round-trip
python3 host/run_cdc_demo.py cfg-load --counter 1
python3 host/run_cdc_demo.py cdc-mode --counter 0

# Real-time monitor of all four counters
python3 host/run_cdc_demo.py monitor

# The headline demo: NO-CDC + auto-increment, sweep the clock slow ->
fast
python3 host/run_cdc_demo.py watch-fail --counter 2

# Force a specific port instead of autodetect
python3 host/run_cdc_demo.py --port /dev/ttyUSB1 smoke
```

Each subcommand prints a human-readable result and returns a non-zero exit code on failure, so the CLI doubles as a board bring-up smoke test.

5.3.1 The "watch it fail" procedure

1. Program the board (`make program-demo`).
2. `python3 host/run_cdc_demo.py watch-fail --counter 2` sets counter 2 to NO-CDC + auto-increment, sets `DISP_SELECT` to 2, and sweeps `div_pickoff` from slow to fast, sampling `VALUE` at each step.
3. Read the on-board 7-seg: clean counting at slow pickoffs, visible scramble at fast pickoffs. Compare against a counter left in a safe mode — it stays clean.

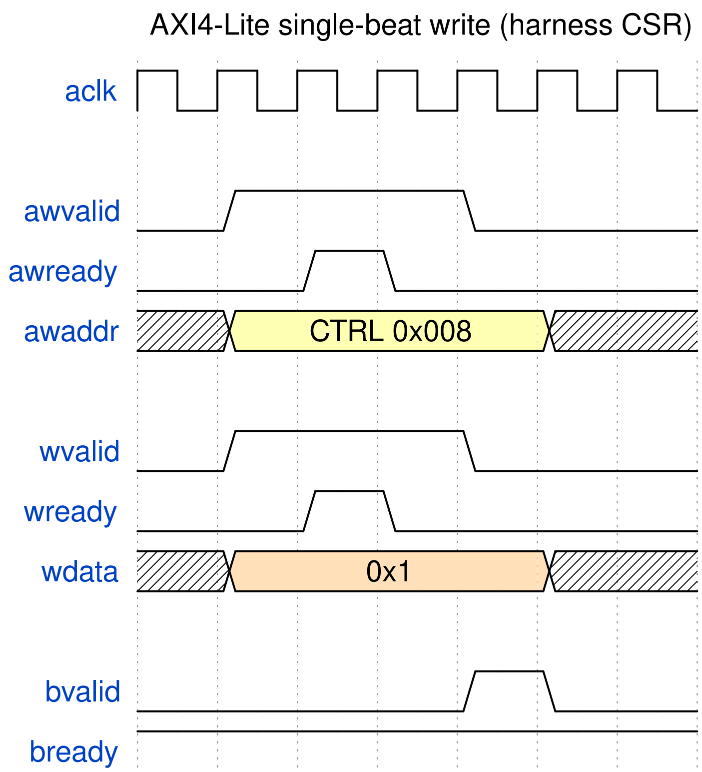
6 Harness CSR Configuration

The `cdc_demo_harness` is a single flat AXI4-Lite slave. The host reaches it through the UART bridge at **harness base `0x0`**. Every register is 32 bits, word-addressable.

Registers are accessed by name, never by offset. The layout is authoritative in `rtl/cdc_demo_harness.sv`, described in `rtl/cdc_demo_csr.rdl`, and compiled to `dv/tbclasses/cdc_demo_csr_regmap.py` (PeakRDL). The driver, the host programs, and the sim all resolve names through that regmap; the `consistency` test guards the three against drift.

Every named write becomes a single AXI4-Lite beat that the UART bridge drives into the harness (a named read is the mirror AR/R exchange):

6.0.0.1 Waveform 5.1: AXI4-Lite Single-Beat Write



AXI4-Lite single-beat write

Source: 01_axil_write.json

6.1 Global registers

Global CSR registers — register / field / offset / bit slice

Register	Offset	Field	Bits	Access	Description
`BUILD_ID`	0x000	`value`	[31:0]	R	ASCII “CDC1” (0x43434331) — read first to confirm the

Register	Offset	Field	Bits	Access	Description
					bitstream
`STATUS`	0x004	`alive0–alive3`	[3:0]	R	Counter *i* toggled within the last ~1 s window
`STATUS`	0x004	`uart_rx`	[4]	R	UART RX activity
`STATUS`	0x004	`uart_tx`	[5]	R	UART TX activity
`STATUS`	0x004	`any_written`	[6]	R	Any CSR written since reset
`STATUS`	0x004	`reset_ok`	[31]	R	Reset deasserted / harness alive
`CTRL`	0x008	`soft_reset`	[0]	RW	Pulse: full sys_clk reset
`CTRL`	0x008	`freeze`	[1]	RW	Freeze all counters
`CTRL`	0x008	`ignore_btn`	[2]	RW	Ignore physical buttons (host-press only)
`DISP_SELECT`	0x00C	`sel`	[1:0]	RW	Which counter drives the 7-seg
`SCRATCH`	0x010	`value`	[31:0]	RW	Link-sanity scratch (write, read back)

6.2 Per-counter block

Counter `i` (0–3) occupies `0x040 + i*0x40`. The regmap keys are `COUNTER{i}_<REG>` (e.g. `COUNTER2_DIVISOR`).

*Per-counter CSR block — counter i absolute offset = 0x40 + i*0x40 + column*

Register	+Offset	Field	Bits	Access	Description
`DIVISOR`	+0x00	`clock_select`	[2:0]	RW	Clock source (0–3 MMCM, 4 = divided)
`DIVISOR`	+0x00	`div_pickoff`	[12:8]	RW	Divided-clock pickoff (when

Register	+Offset	Field	Bits	Access	Description
					`clock_select=4`)
`INIT`	+0x04	`value`	[15:0]	RW	Value loaded by `CFG_LOAD`
`INCREMENT`	+0x08	`value`	[15:0]	RW	Advance amount per press
`CFG_LOAD`	+0x0C	`strobe`	[0]	W	Pulse: load `INIT` into the counter
`HOST_PRESS`	+0x10	`strobe`	[0]	W	Pulse: inject one virtual press
`VALUE`	+0x14	`value`	[15:0]	R	Current value, CDC'd per `CDC_MODE`
`PRESS_COUNT`	+0x18	`value`	[15:0]	R	Debounced press count (always Gray-CDC'd)
`CTR_CLK_TICKS`	+0x1C	`value`	[31:0]	R	Free-running `ctr_clk` tick count
`CDC_MODE`	+0x20	`mode`	[2:0]	RW	CDC strategy for `VALUE` (0–4; see Chapter 2)
`AUTO_INC`	+0x24	`en`	[0]	RW	1 = advance every `ctr_clk` edge

6.3 By-name access

The driver (`host/cdc_demo.py`) wraps `UartRegisterMap`. Whole-register and individual-field access both work:

```
from cdc_demo import CdcDemoDriver
d = CdcDemoDriver(port="/dev/ttyUSB1")          # or bridge=... for sim

d.build_id()                                    # -> 0x43434331
d.scratch(0xDEADBEEF)                            # write + read back
d.disp_select(2)                                 # DISP_SELECT.sel = 2

c = d.counter(2)
```

```
c.set_cdc_mode(2)                              #
COUNTER2_CDC_MODE.mode = 2
c.set_div_pickoff(23)                          # DIVISOR.
{clock_select=4, div_pickoff=23}
c.set_init(0x10); c.set_increment(3); c.load()   # load INIT
c.press()                                      # HOST_PRESS
c.value(), c.press_count()                     # CDC'd status
readback
```

Packed fields are spliced by name with a read-modify-write, so setting `div_pickoff` never disturbs `clock_select`. Write-only strobes (`CFG_LOAD`, `HOST_PRESS`) are writable by name but read back as 0 by design.

6.4 Regenerating the regmap

`rtl/cdc_demo_harness.sv` is hand-written; the RDL is a descriptor of it. After editing either the SV or the RDL:

```
make regmap        # regenerate dv/tbclasses/cdc_demo_csr_regmap.py
from the RDL
make consistency   # assert regmap == SV (globals + reconstructed per-
counter)
```

Never hand-edit the generated regmap — it is regenerated from the RDL.

7 Simulation / Silicon Equivalence

This project follows the shared Nexys A7 UART-characterization methodology (`bin/TBClasses/harness/`): **one authored-once host program runs byte-for-byte identically on the FPGA and in a cocotb simulation.** The equivalence boundary is the ASCII `W`/`R` byte stream on the UART wire — if the same program emits the same bytes, the same `uart_axil_bridge` RTL decodes them the same way on silicon and in sim.

7.1 The layered stack (only the bottom layer differs)

```
Programs   cdc_programs.py            authored once, unchanged
FPGA/sim
Driver     CdcDemoDriver + UartRegisterMap   by-name registers
Protocol   UARTAxiBridge (W/R ASCII)  same code both sides
-- byte stream
---------------------------------------------------------
Channel    SerialChannel (pyserial)  |  CocotbUartChannel (cocotb)
Wire       FTDI/USB UART             |  UARTMaster/Monitor on DUT pins
```

The bridge is **injectable**: `CdcDemoDriver(port=...)` opens a real serial port; `CdcDemoDriver(bridge=UARTAxiBridge(channel=cocotb_channel))` drives the sim. Nothing above the wire forks.

7.2 How the sim runs the same program

`dv/tb/cdc_demo_uart_tb_top.sv` wraps the **real** `uart_axil_bridge` + `cdc_demo_harness` + four `cdc_counter_domain` instances. `dv/tests/test_cdc_demo_uart.py`:

1. Starts `aclk` and reset, and drives the four `ctr_clk[i]` at co-prime periods.
2. Builds a `CocotbUartChannel` via `make_uart_channel(dut, dut.aclk, CLKS_PER_BIT)`.
3. Constructs `CdcDemoDriver(bridge=UARTAxiBridge(channel=chan))`.
4. Runs the unmodified `cdc_programs.*` functions inside `cocotb.external(...)`.

The synchronous host program runs in a worker thread; the channel's read/write are `cocotb.function` wrappers that drive the UART master and advance simulation time. The four cocotb tests (smoke, press, cfg_load, cdc_mode) assert the same results the CLI checks on the board.

7.3 What the sim can and cannot reproduce

- **CAN:** everything digital — the bridge RTL, CSR decode, per-counter CDC datapaths, register logic. The sim proves the byte protocol and the harness contract before you ever touch the board.
- **CANNOT:** the analog clock tree. `cdc_demo_top`'s `MMCME2_BASE` / `BUFGMUX_CTRL` / `IBUF` / `BUFG` do not simulate in Verilator, so the tb_top drives `ctr_clk[i]` behaviorally instead. Consequently the NO-CDC "garbage read" is a silicon-only effect — in Verilator (no metastability) mode 0 reads the true value, so the sim asserts exact values in NO-CDC while the board shows scramble at speed.

7.4 Gotchas worth keeping

- **Lower `CLKS_PER_BIT` in sim** (16, in both the tb_top param and the BFM) — bit-timing only, the byte stream is unchanged, so equivalence holds.
- **Wrap the full harness, not the inner block** — the real `uart_axil_bridge` + `cdc_demo_harness` must be in the DUT, or the two flows are only "vaguely similar," not equivalent.
- **Use `cocotb.function`, not a free-running pump** — a pump coroutine plus thread queues stalls the scheduler; `cocotb.function` is what steps the sim from worker-thread calls.

8 Troubleshooting

8.1 The host CLI cannot find the board

The USB-UART re-enumerates across reboots and replugs, so the port number drifts. The CLI defaults to `--port auto`, which probes every `/dev/ttyUSB*` and keeps the one that answers `BUILD_ID == 0x43434331` ("CDC1"). If autodetect fails:

- Confirm the board is powered and programmed with the **phase-2** bitstream (`make program-demo`), not phase 1.
- Check the FTDI cable and that no other program holds the port.
- Force a specific device with `--port /dev/ttyUSB1`.

8.2 Values look scrambled / never settle

Expected in **NO-CDC mode at a fast clock** — that is the demonstration. For coherent reads, put the counter in a safe `CDC_MODE` (2 = SYNC-FIFO, or 3/4 for the handshakes) before reading `VALUE`. `PRESS_COUNT` always uses Gray-coded CDC and is coherent regardless of mode.

8.3 make `lint-demo` — Xilinx primitive errors

`cdc_demo_top` instantiates `MMCME2_BASE` / `BUFGMUX_CTRL` / `IBUF` / `BUFG`, which Verilator cannot find. `rtl/verilator_xilinx_stubs.sv` provides `` `ifdef VERILATOR``-guarded pass-through stubs (Vivado uses the real unisims at synthesis; the stub file is never in a Vivado flow). If lint fails with "Cannot find module" for one of these, confirm the stub file is first in the `lint-demo` source list.

8.4 make `sim` (phase 1) fails to compile

Phase 1 uses `clock_divider.sv`, which relies on the `` `ALWAYS_FF_RST`` macro from `rtl/amba/includes/reset_defs.svh`. The phase-1 test compiles that header first and adds `+incdir+rtl/amba/includes`. If you see "Define or directive not defined: `ALWAYS_FF_RST`", that include setup is missing.

8.5 make `consistency` fails after editing registers

The generated regmap drifted from the hand-written SV. Regenerate and re-check:

```
make regmap
make consistency
```

If it still fails, the SV header table or the `CTR_OFF_*` localparams in `cdc_demo_harness.sv` disagree with `rtl/cdc_demo_csr.rdl` — reconcile the three.

8.6 Sim runs but nothing advances

If a UART-equivalence test hangs, the transport is almost certainly using a pump instead of `cocotb.function`, or `CLKS_PER_BIT` was left at the silicon value (868), making each command hundreds of thousands of clocks. Both are covered in Chapter 6.